Rotation About an Arbitrary Point

C++ program using graphics.h, with sample input and print-friendly output

```
#include <graphics.h>
#include <iostream>
#include <cmath>
using namespace std;

struct Point {
    double x, y;
};

void drawPolygon(Point pts[], int count, int color) {
    setcolor(color);
    for (int i = 0; i < count; i++) {
        line((int)pts[i].x, (int)pts[i].y,
             (int)pts[(i + 1) % count].x,
             (int)pts[(i + 1) % count].y);
    }
}

void rotateAboutArbitraryPoint(Point pts[], int count,
                               double angle, Point pivot) {
    const double PI = 3.14159265358979323846;
    double rad = angle * PI / 180.0;
    double cosA = cos(rad);
    double sinA = sin(rad);

    for (int i = 0; i < count; i++) {
        double x_rel = pts[i].x - pivot.x;
        double y_rel = pts[i].y - pivot.y;

        pts[i].x = pivot.x + (x_rel * cosA - y_rel * sinA);
        pts[i].y = pivot.y + (x_rel * sinA + y_rel * cosA);
    }
}

int main() {
    double angle, px, py;

    cout << "Rotation about an Arbitrary Point\\n";
    cout << "Enter angle of rotation (degrees): ";
    cin >> angle;
    cout << "Enter arbitrary pivot point (px py): ";
    cin >> px >> py;

    initwindow(800, 600, "Rotation about Arbitrary Point");

    Point rect[4] = {
        {300, 200}, {450, 200}, {450, 300}, {300, 300}
    };
    Point original[4];
    for (int i = 0; i < 4; i++) original[i] = rect[i];

    drawPolygon(original, 4, WHITE);
    setcolor(LIGHTGRAY);
    line(0, 300, 800, 300);

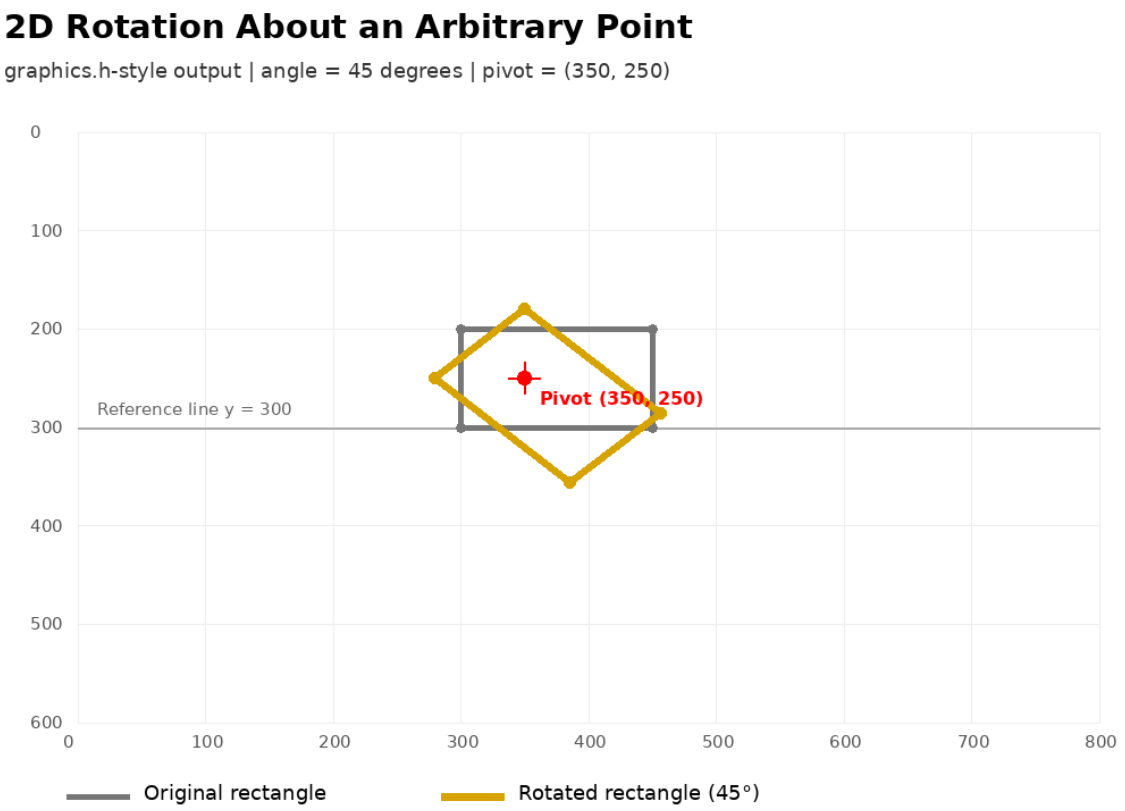
    Point pivot = {px, py};
    rotateAboutArbitraryPoint(rect, 4, angle, pivot);
    drawPolygon(rect, 4, YELLOW);

    setcolor(RED);
    circle((int)pivot.x, (int)pivot.y, 4);

    getch();
    closegraph();
    return 0;
}
```

Sample Graphics Output

Input: Rotation choice = 2; angle = 45°; pivot point = (350, 250).



Original rectangle: (300,200), (450,200), (450,300), (300,300). The yellow rectangle is the rotated result; the red mark shows the pivot point.

Note: This is a clean print-friendly illustration of the expected graphics output, not a screenshot captured from a running graphics.h program.